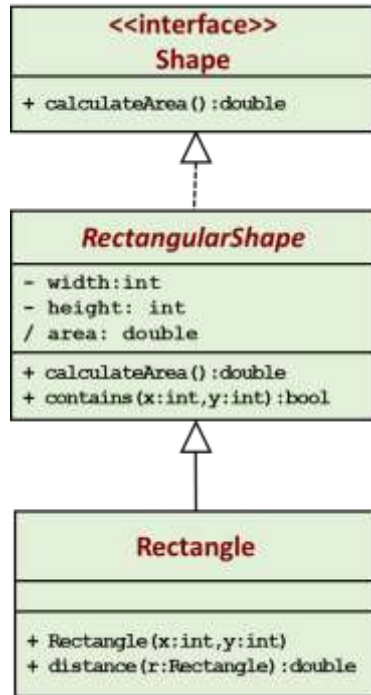
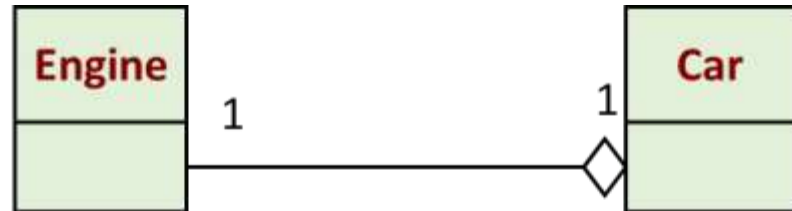


Creational Patterns

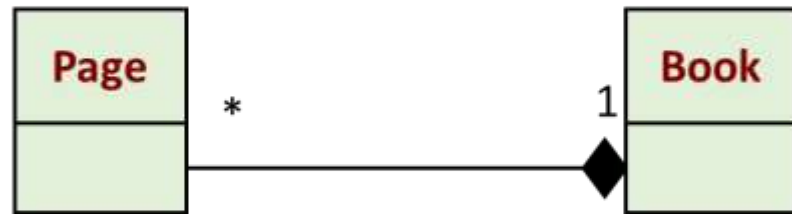
UML Revisited



Generalization



Aggregation: "is part of"



Composition: "is entirely made of"



Dependency: "uses"

Creational Patterns to be Covered

- Factory method
- Abstract factory
- Builder
- Singleton

Examples taken from:

<https://refactoring.guru/design-patterns>

<https://javatechonline.com/java-design-patterns-java/>

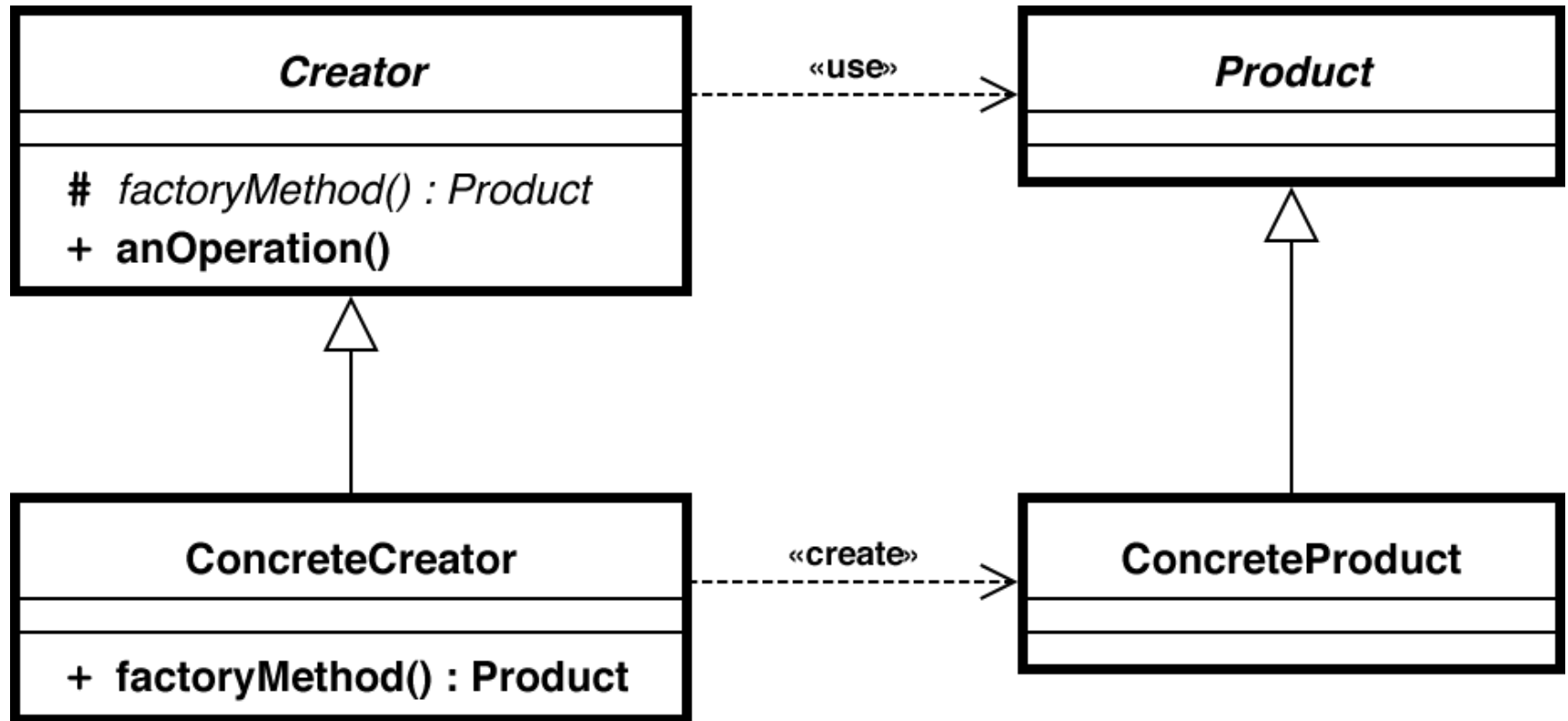
https://www.tutorialspoint.com/design_pattern/

<https://www.javatpoint.com/design-patterns-in-java>

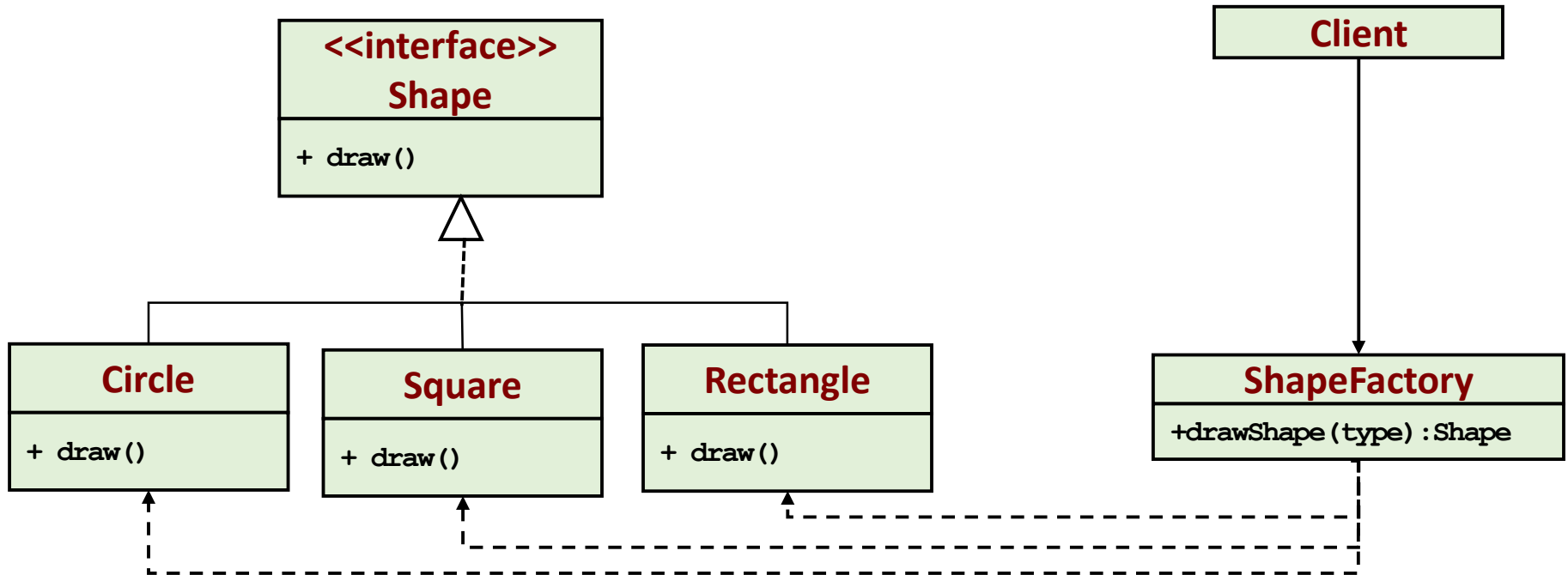
Factory Method

Intent	• Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.
Problem	• A framework needs to standardize the architectural model for a range of applications, but allow for individual applications to define their own domain objects and provide for their instantiation. • Enable the creator to defer product creation to sub-class.

Solution



Example: Drawing



```

public interface Shape {
    void draw();
}
public class Rectangle implements Shape {
    @Override
    public void draw() {
        System.out.println("Inside Rectangle::draw() method.");
    }
}

```

Similar implementation for **Square** and **Circle**

```

public class ShapeFactory {

    //use getShape method to get object of type shape
    public Shape getShape(String shapeType){
        if(shapeType == null){
            return null;
        }
        if(shapeType.equalsIgnoreCase("CIRCLE")){
            return new Circle();
        } else if(shapeType.equalsIgnoreCase("RECTANGLE")){
            return new Rectangle();
        } else if(shapeType.equalsIgnoreCase("SQUARE")){
            return new Square();
        }
        return null;
    }
}

```

```

public class FactoryPatternDemo {

    public static void main(String[] args) {
        ShapeFactory shapeFactory = new ShapeFactory();

        //get an object of Circle and call its draw method.
        Shape shape1 = shapeFactory.getShape("CIRCLE");

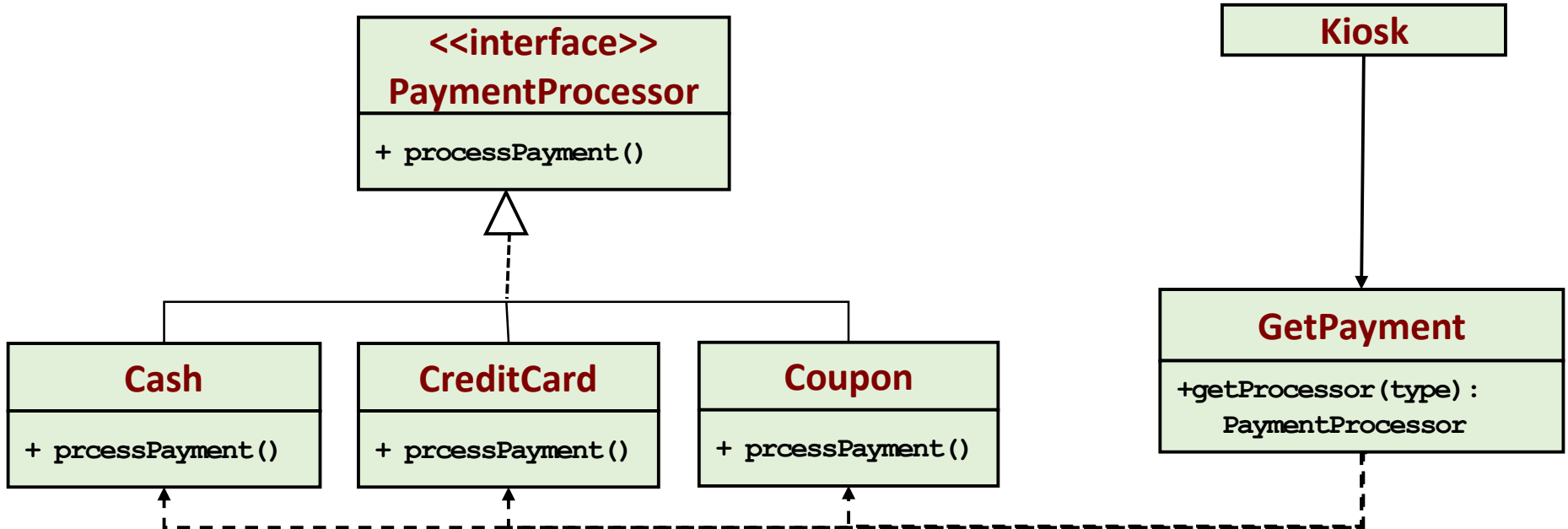
        //call draw method of Circle
        shape1.draw();

        //get an object of Rectangle and call its draw method.
        Shape shape2 = shapeFactory.getShape("RECTANGLE");

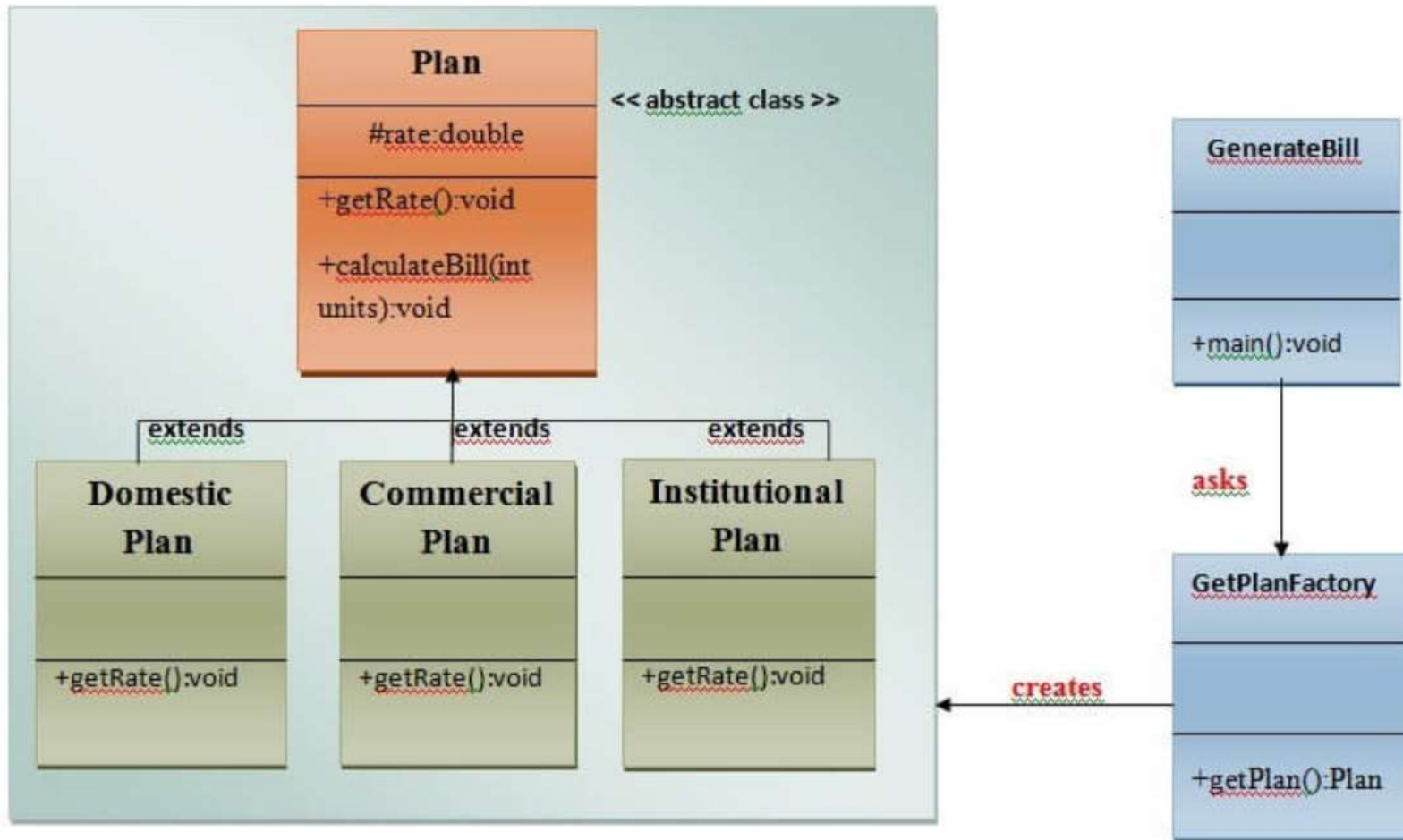
        //call draw method of Rectangle
        shape2.draw();
    }
}

```

Example: Kiosk



Factory



```

import java.io.*;
abstract class Plan{
    protected double rate;
    abstract void getRate();

    public void calculateBill(int units){
        System.out.println(units*rate);
    }
} //end of Plan class.

```

Step 2: Create the concrete classes that extends Plan

```

class DomesticPlan extends Plan{
    //@override
    public void getRate(){
        rate=3.50;
    }
} //end of DomesticPlan class.

```

```

class CommercialPlan extends Plan{
    //@override
    public void getRate(){
        rate=7.50;
    }
} //end of CommercialPlan class.

```

```

class InstitutionalPlan extends Plan{
    //@override
    public void getRate(){
        rate=5.50;
    }
} //end of InstitutionalPlan class.

```

```

class GetPlanFactory{

    //use getPlan method to get object of type Plan
    public Plan getPlan(String planType){
        if(planType == null){
            return null;
        }
        if(planType.equalsIgnoreCase("DOMESTICPLAN")) {
            return new DomesticPlan();
        }
        else if(planType.equalsIgnoreCase("COMMERCIALPLAN")){
            return new CommercialPlan();
        }
        else if(planType.equalsIgnoreCase("INSTITUTIONALPLAN")) {
            return new InstitutionalPlan();
        }
        return null;
    }
} //end of GetPlanFactory class.

```

```

import java.io.*;

class GenerateBill{

    public static void main(String args[])throws IOException{
        GetPlanFactory planFactory = new GetPlanFactory();

        System.out.print("Enter the name of plan for which the bill will be generated: ");
        BufferedReader br=new BufferedReader(new InputStreamReader(System.in));

        String planName=br.readLine();
        System.out.print("Enter the number of units for bill will be calculated: ");
        int units=Integer.parseInt(br.readLine());

        Plan p = planFactory.getPlan(planName);
        //call getRate() method and calculateBill()method of DomesticPlan.

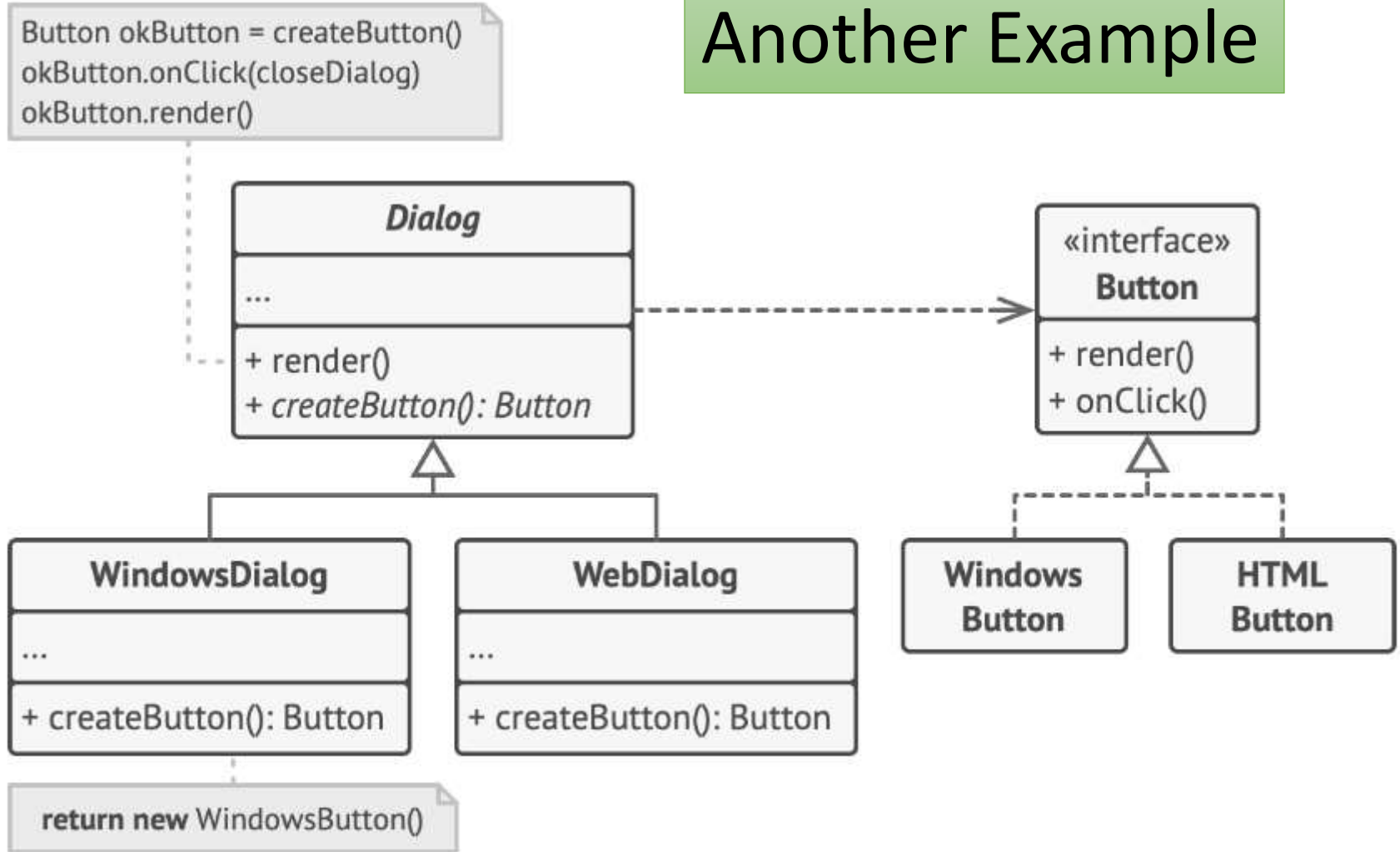
        System.out.print("Bill amount for "+planName+" of "+units+" units is: ");
        p.getRate();
        p.calculateBill(units);
    }
} //end of GenerateBill class.

```

Another Example Usecase

- Imagine that you're creating a logistics management application. The first version of your app can only handle transportation by trucks, so the bulk of your code lives inside the Truck class.
- After a while, your app becomes pretty popular. Each day you receive dozens of requests from sea transportation companies to incorporate sea logistics into the app.
- Great news, right? But how about the code? At present, most of your code is coupled to the Truck class. Adding Ships into the app would require making changes to the entire codebase. Moreover, if later you decide to add another type of transportation to the app, you will probably need to make all of these changes again.
- As a result, you will end up with pretty nasty code, riddled with conditionals that switch the app's behavior depending on the class of transportation objects.
- Solution: <https://refactoring.guru/design-patterns/factory-method>

Another Example



The cross-platform dialog example.

<https://refactoring.guru/design-patterns/factory-method>

Use the Factory Method when you want to provide users of your library or framework with a way to extend its internal components

- Imagine that you write an app using an open source UI framework.
- Your app should have round buttons, but the framework only provides square ones.
- You extend the standard Button class with a glorious RoundButton subclass.
- But now you need to tell the main UIFramework class to use the new button subclass instead of a default one.
- To achieve this, you create a subclass UIWithRoundButtons from a base framework class and override its createButton method.
- While this method returns Button objects in the base class, you make your subclass return RoundButton objects.
- Now use the UIWithRoundButtons class instead of UIFramework.

Consequences/Benefits

- Factory design pattern provides approach to code for interface rather than implementation.
- Factory pattern removes the instantiation of actual implementation classes from client code. Factory pattern makes our code more robust, less coupled and easy to extend. For example, we can easily change lower class implementation because client program is unaware of this.
- Factory pattern provides abstraction between implementation and client classes through inheritance.

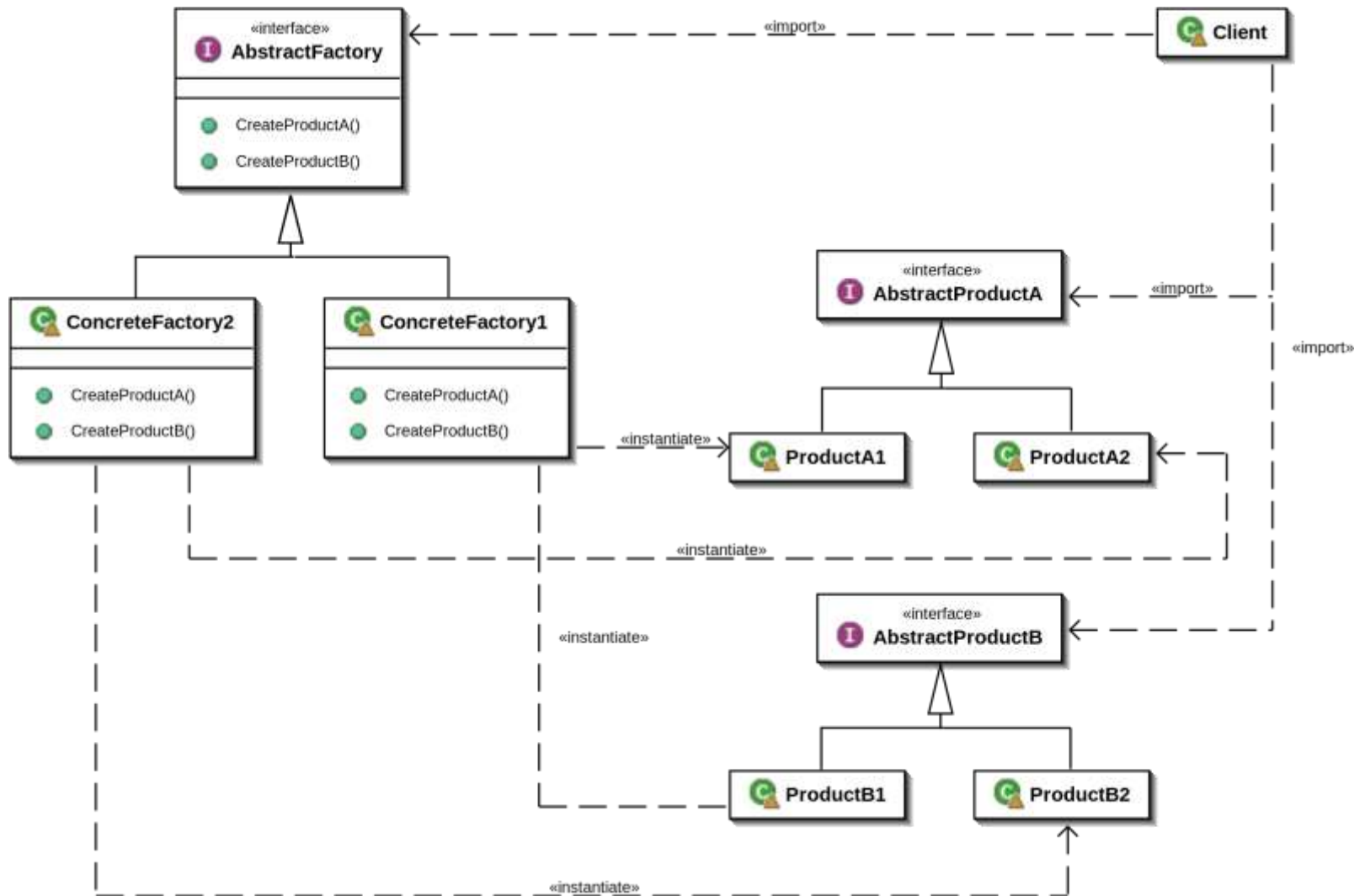
Pros and Cons

- ✓ You avoid tight coupling between the creator and the concrete products.
- ✓ *Single Responsibility Principle.* You can move the product creation code into one place in the program, making the code easier to support.
- ✓ *Open/Closed Principle.* You can introduce new types of products into the program without breaking existing client code.
- ✗ The code may become more complicated since you need to introduce a lot of new subclasses to implement the pattern. The best case scenario is when you're introducing the pattern into an existing hierarchy of creator classes.

Abstract Factory

Intent	• Provide an interface for creating families of related or dependent objects without specifying their concrete classes.
Problem	• A portable application needs to encapsulate platform dependencies • Consider an application that support multiple look and feels. • An application need to work with multiple types of DBMS

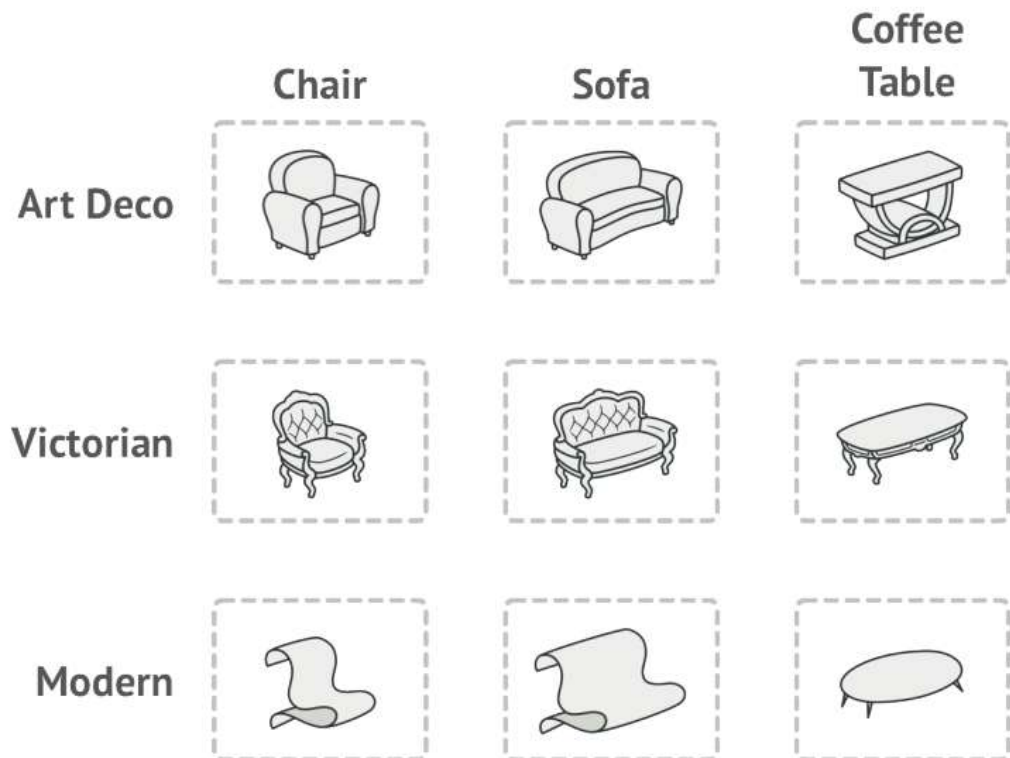
Solution



Problem

- Imagine that you're creating a furniture shop simulator. Your code consists of classes that represent a family of products and their variants.

<https://refactoring.guru/design-patterns/abstract-factory>



Product families and their variants.

Abstract Factory

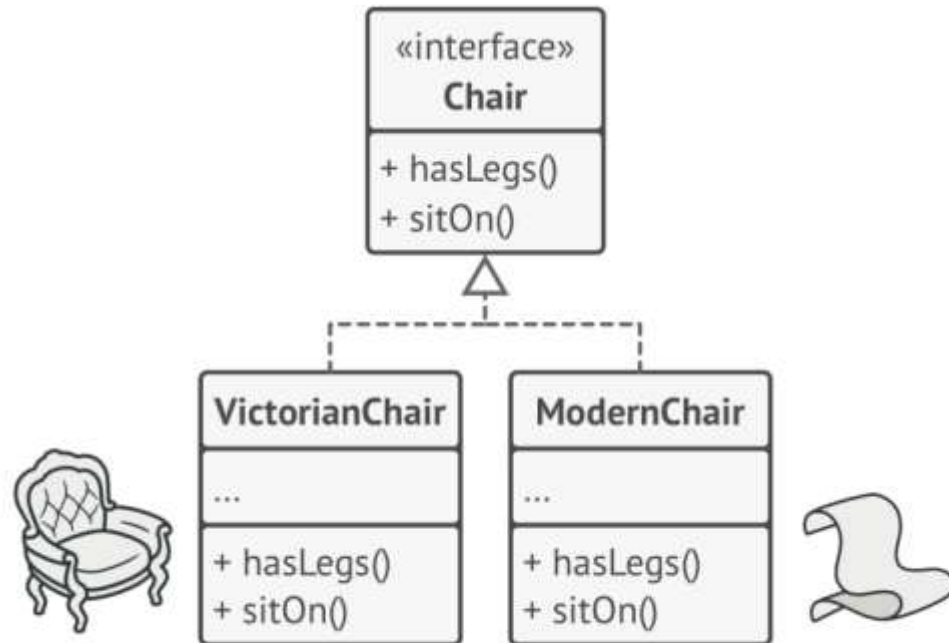
- You need a way to create individual furniture objects so that they match other objects of the same family.
- Customers get quite mad when they receive non-matching furniture.
- Also, you don't want to change existing code when adding new products or families of products to the program.
- Furniture vendors update their catalogs very often, and you wouldn't want to change the core code each time it happens.



A Modern-style sofa doesn't match Victorian-style chairs.

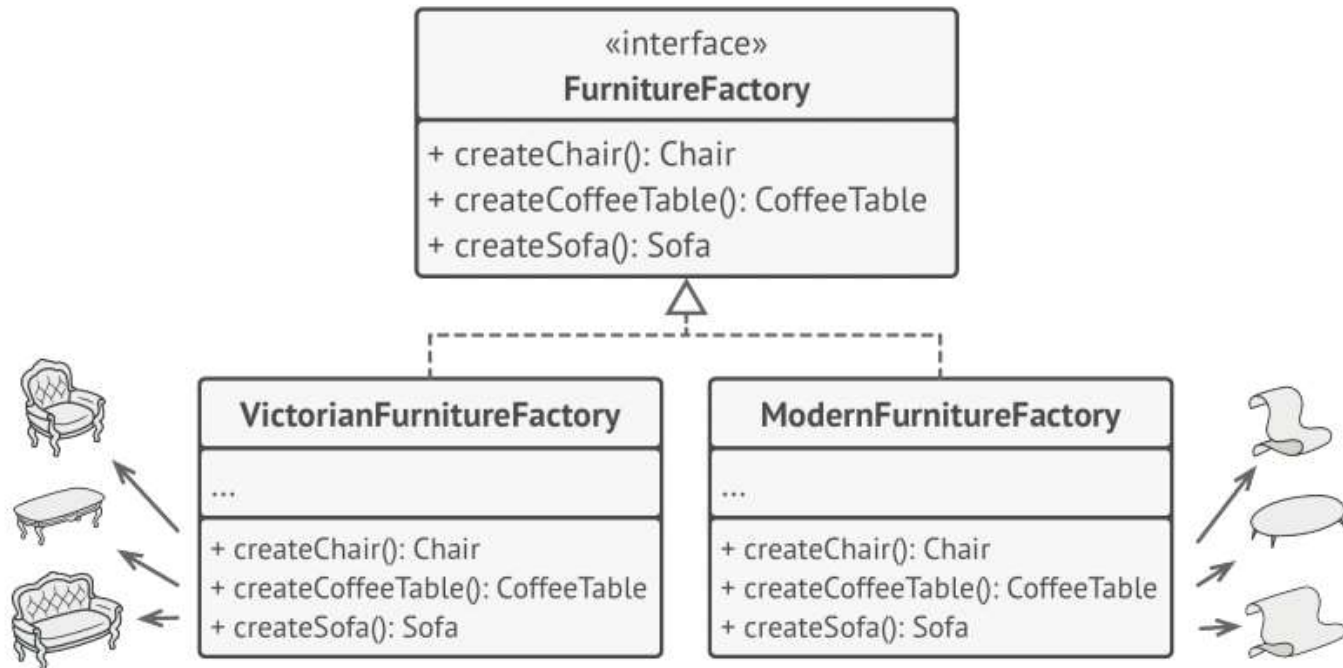
Solution

- The first thing the Abstract Factory pattern suggests is to explicitly declare interfaces for each distinct product of the product family (e.g., chair, sofa or coffee table).
- Then you can make all variants of products follow those interfaces. For example, all chair variants can implement the Chair interface; all coffee table variants can implement the CoffeeTable interface, and so on.



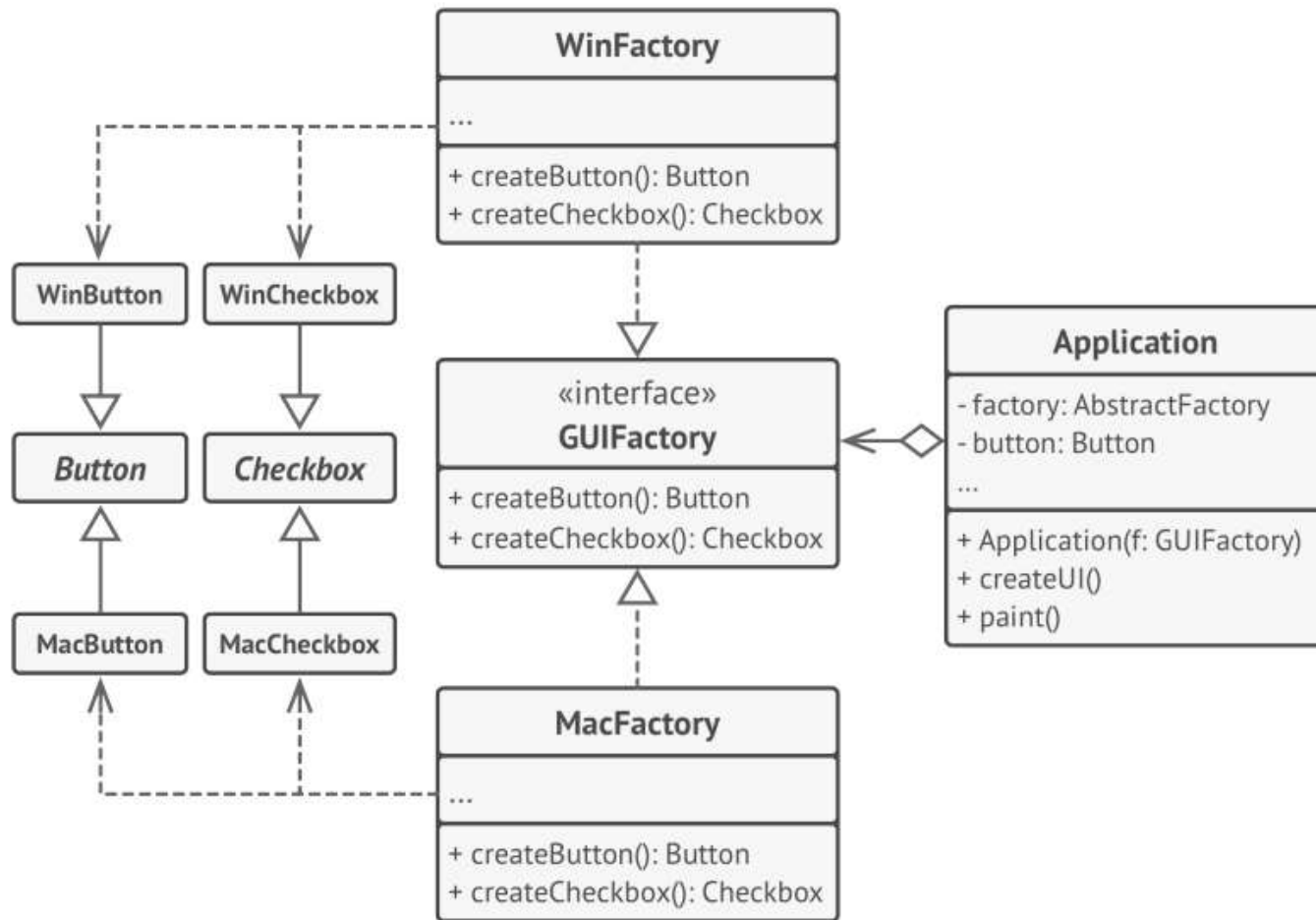
All variants of the same object must be moved to a single class hierarchy.

- The next move is to declare the *Abstract Factory*—an interface with a list of creation methods for all products that are part of the product family (for example, createChair).
- For each variant of a product family, we create a separate factory class based on the AbstractFactory interface.
- A factory is a class that returns products of a particular kind. For example, the ModernFactory can only create ModernChair, ModernSofa and ModernCoffeeTable objects.



Each concrete factory corresponds to a specific product variant.

Another Example



The cross-platform UI classes example.

Another Example

- When an application launches, it checks the type of the current operating system.
- The app uses this information to create a factory object from a class that matches the operating system.
- The rest of the code uses this factory to create UI elements.
- With this approach, the client code doesn't depend on concrete classes of factories and UI elements as long as it works with these objects via their abstract interfaces.
- This also lets the client code support other factories or UI elements that you might add in the future.

```
interface GUIFactory is
    method createButton():Button
    method createCheckbox():Checkbox
```

```
class WinFactory implements GUIFactory is
    method createButton():Button is
        return new WinButton()
    method createCheckbox():Checkbox is
        return new WinCheckbox()
```

// Each concrete factory has a corresponding product variant.

```
class MacFactory implements GUIFactory is
    method createButton():Button is
        return new MacButton()
    method createCheckbox():Checkbox is
        return new MacCheckbox()
```

interface **Button** is

method `paint()`

// Concrete products are created by corresponding concrete
// factories.

class **WinButton** implements **Button** is

method `paint()` is

// Render a button in Windows style.

class **MacButton** implements **Button** is

method `paint()` is

// Render a button in macOS style.

```
interface Checkbox is
```

```
    method paint()
```

```
class WinCheckbox implements Checkbox is
```

```
    method paint() is
```

```
        // Render a checkbox in Windows style.
```

```
class MacCheckbox implements Checkbox is
```

```
    method paint() is
```

```
        // Render a checkbox in macOS style.
```

```
// The client code works with factories and products only  
// through abstract types: GUIFactory, Button and Checkbox. This  
// lets you pass any factory or product subclass to the client  
// code without breaking it.
```

class Application is

private field button: Button

constructor Application(factory: GUIFactory) **is**

this.factory = factory

method createUI() **is**

this.button = factory.createButton()

method paint() **is**

button.paint()

```
// The application picks the factory type depending on the  
// current configuration or environment settings and creates it  
// at runtime (usually at the initialization stage).
```

```
class ApplicationConfigurator is
```

```
  method main() is
```

```
    config = readApplicationConfigFile()
```

```
    if (config.OS == "Windows") then
```

```
        factory = new WinFactory()
```

```
    else if (config.OS == "Mac") then
```

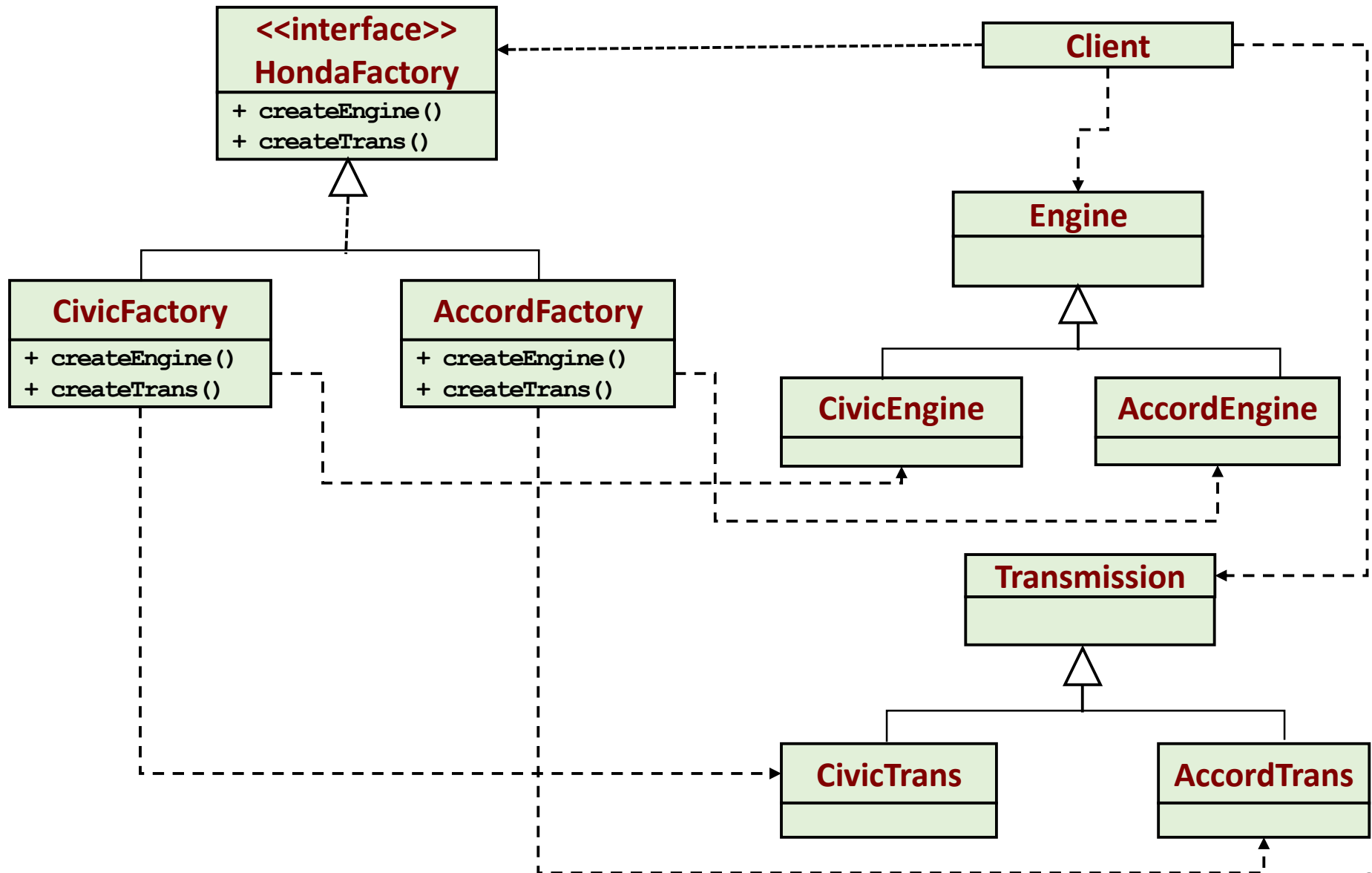
```
        factory = new MacFactory()
```

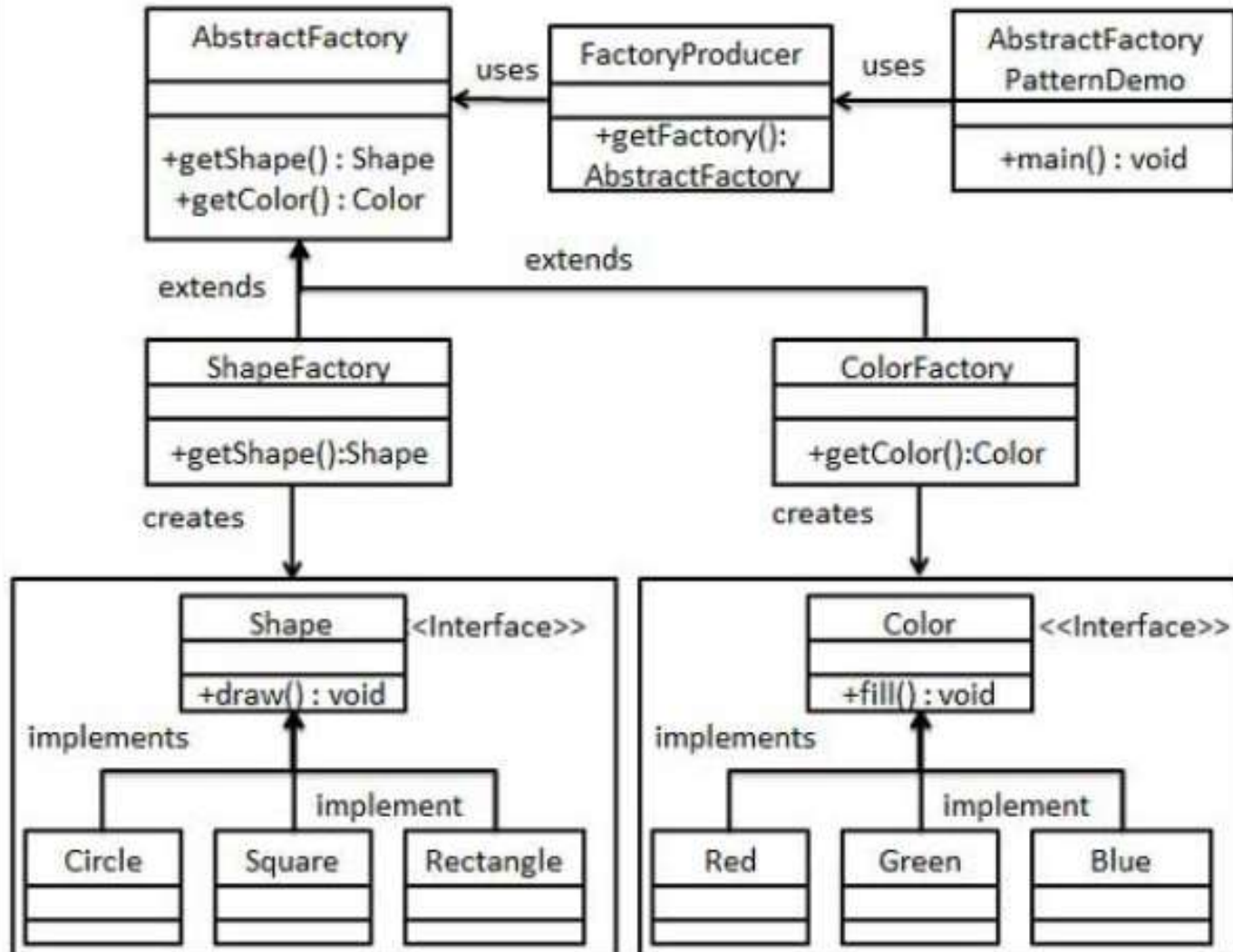
```
    else
```

```
        throw new Exception("Error! Unknown operating system.")
```

```
    Application app = new Application(factory)
```

Example: Car Factory





```
public abstract class AbstractFactory {  
    abstract Color getColor(String color);  
    abstract Shape getShape(String shape) ;  
}
```

```
public class ShapeFactory extends AbstractFactory {  
  
    @Override  
    public Shape getShape(String shapeType){  
        if(shapeType == null){  
            return null;  
        }  
        if(shapeType.equalsIgnoreCase("CIRCLE")){  
            return new Circle();  
        } else if(shapeType.equalsIgnoreCase("RECTANGLE")){  
            return new Rectangle();  
        } else if(shapeType.equalsIgnoreCase("SQUARE")){  
            return new Square();  
        }  
        return null;  
    }  
}
```

```
public class FactoryProducer {  
    public static AbstractFactory getFactory(String choice){  
        if(choice.equalsIgnoreCase("SHAPE")){  
            return new ShapeFactory();  
        } else if(choice.equalsIgnoreCase("COLOR")){  
            return new ColorFactory();  
        }  
        return null;  
    }  
}
```

```
public class AbstractFactoryPatternDemo {  
    public static void main(String[] args) {  
  
        //get shape factory  
        AbstractFactory shapeFactory = FactoryProducer.getFactory("SHAPE")  
  
        //get an object of Shape Circle  
        Shape shape1 = shapeFactory.getShape("CIRCLE");  
  
        //call draw method of Shape Circle  
        shape1.draw();  
  
        //get an object of Shape Rectangle  
        Shape shape2 = shapeFactory.getShape("RECTANGLE");  
  
        //call draw method of Shape Rectangle  
        shape2.draw();  
    }  
}
```

Consequences

- Isolates the concrete class
- Makes exchanging product family easy
- Promotes consistency among products
- Abstract Factory design pattern provides approach to code for interface rather than implementation.
- Abstract Factory pattern is “factory of factories” and can be easily extended to accommodate more products.
- Abstract Factory pattern is robust and avoid conditional logic of Factory pattern.
- Supporting new types of products is difficult
- Use the Abstract Factory when your code needs to work with various families of related products, but you don’t want it to depend on the concrete classes of those products—they might be unknown beforehand or you simply want to allow for future extensibility

Pros and Cons

- ✓ You can be sure that the products you're getting from a factory are compatible with each other.
- ✓ You avoid tight coupling between concrete products and client code.
- ✓ *Single Responsibility Principle*. You can extract the product creation code into one place, making the code easier to support.
- ✓ *Open/Closed Principle*. You can introduce new variants of products without breaking existing client code.
- ✗ The code may become more complicated than it should be, since a lot of new interfaces and classes are introduced along with the pattern.

Singleton

Intent	• Ensure a class has only one instance, and provide a global point of access to it. • Encapsulated "just-in-time initialization" or "initialization on first use".
Problem	• Application needs one, and only one, instance of an object. Additionally, lazy initialization and global access are necessary.

Solution

SingletonClass

- instance: SingletonClass

- SingletonClass()

+ getInstance(): SingletonClass

Example: DB Connection Manger

```
public class DbConnection{

    private static DbConnection instance=null;
    private SqlConnection connection;

    private DbConnection() {
        connection = connectToDatabase(dbUser,dbPassword,dbName) ;
    }

    public static getDbConnection() {
        if (instance== null )
            instance = new DbConnection() ;

        return instance;
    }
}
```

```
DbConnection connection=DbConnection.getConnection() ;
```

How to Implement

1. Add a private static field to the class for storing the singleton instance.
2. Declare a public static creation method for getting the singleton instance.
3. Implement “lazy initialization” inside the static method. It should create a new object on its first call and put it into the static field. The method should always return that instance on all subsequent calls.
4. Make the constructor of the class private. The static method of the class will still be able to call the constructor, but not the other objects.
5. Go over the client code and replace all direct calls to the singleton’s constructor with calls to its static creation method.

Applicability

The Singleton pattern solves two problems at the same time, violating the *Single Responsibility Principle*:

1. **Ensure that a class has just a single instance.** Why would anyone want to control how many instances a class has? The most common reason for this is to control access to some shared resource—for example, a database or a file.

Here's how it works: imagine that you created an object, but after a while decided to create a new one. Instead of receiving a fresh object, you'll get the one you already created.

Note that this behavior is impossible to implement with a regular constructor since a constructor call **must** always return a new object by design.

Applicability

2. **Provide a global access point to that instance.** Remember those global variables that you (all right, me) used to store some essential objects? While they're very handy, they're also very unsafe since any code can potentially overwrite the contents of those variables and crash the app.

Just like a global variable, the Singleton pattern lets you access some object from anywhere in the program. However, it also protects that instance from being overwritten by other code.

There's another side to this problem: you don't want the code that solves problem #1 to be scattered all over your program. It's much better to have it within one class, especially if the rest of your code already depends on it.

Unlike global variables, the Singleton pattern guarantees that there's just one instance of a class. Nothing, except for the Singleton class itself, can replace the cached instance

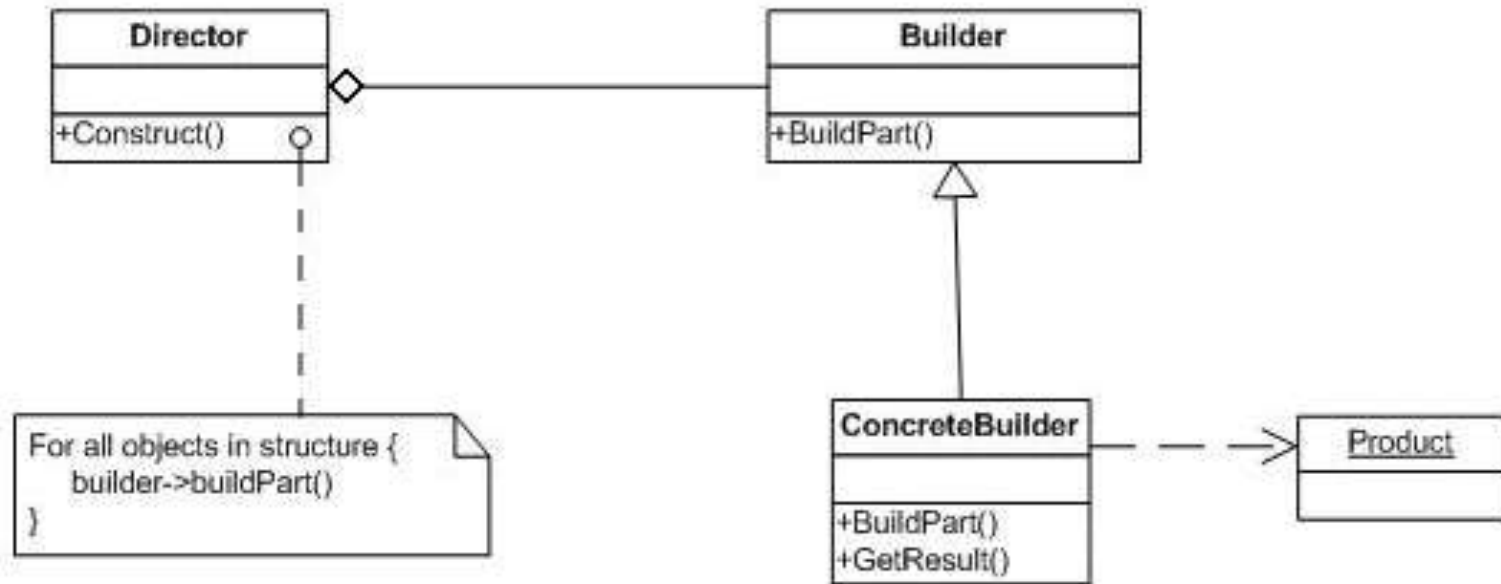
Consequences

- Controlled access to sole instance
- Reduced namespace
- Permits variable number of instances

Builder

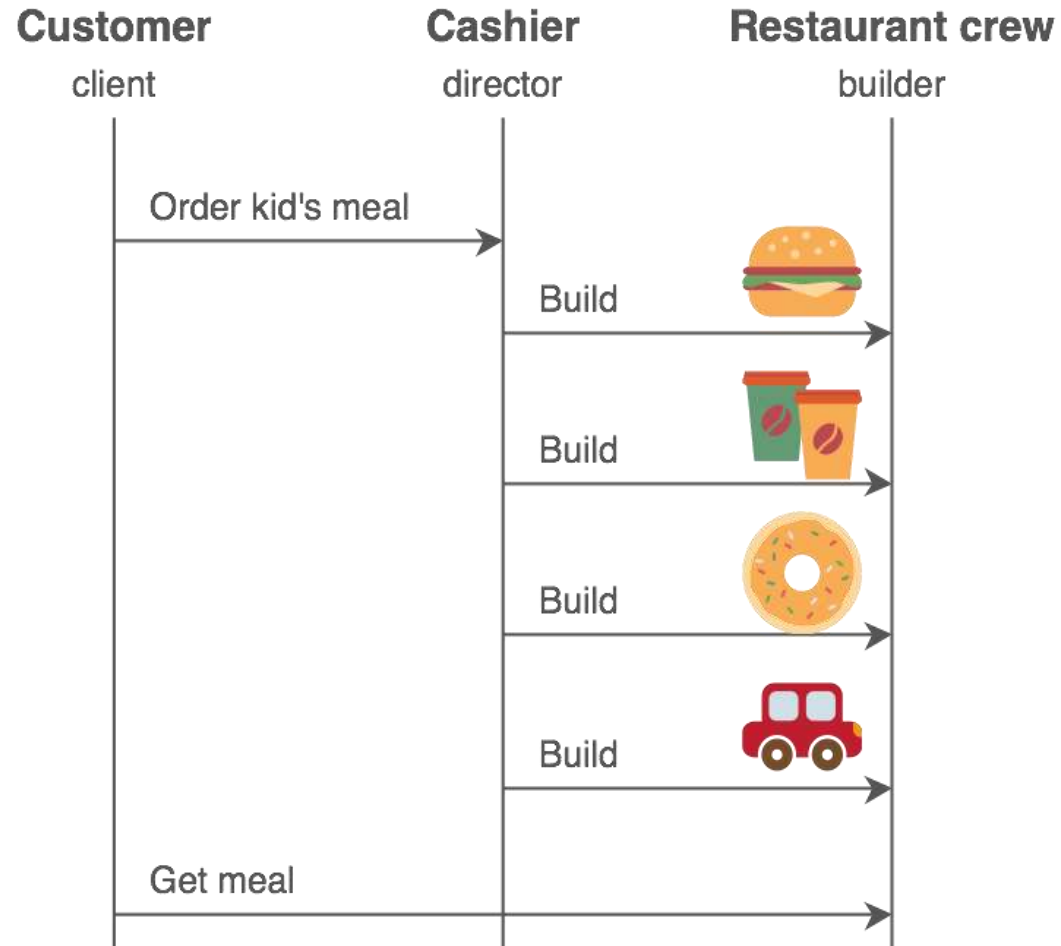
Intent	• Separate the construction of a complex object from its representation so that the same construction process can create different representations. • Parse a complex representation, create one of several targets.
Problem	• An application needs to create the elements of a complex aggregate. The specification for the aggregate exists on secondary storage and one of many representations needs to be built in primary storage.

Solution



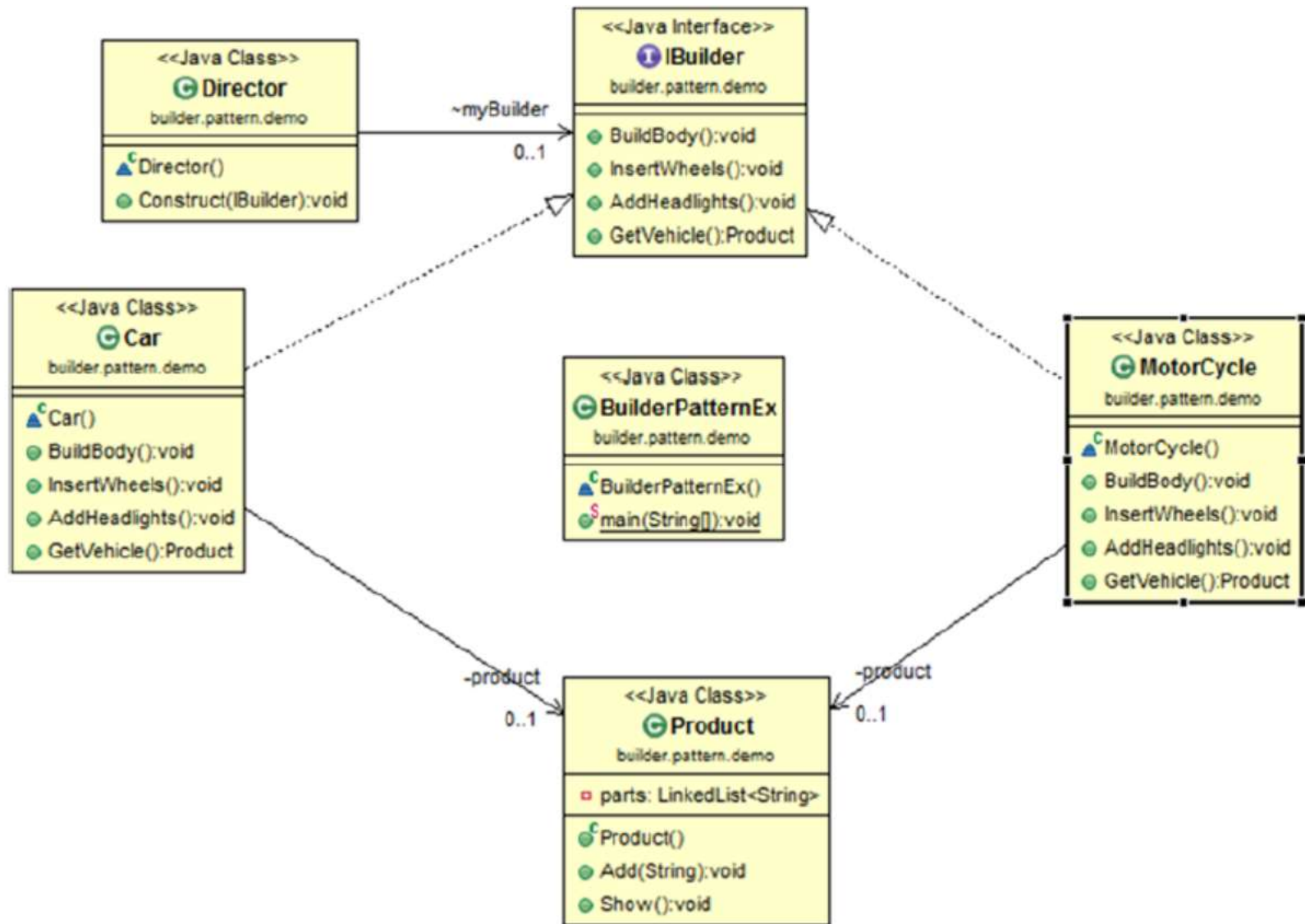
Examples

To create a computer, different parts are assembled depending upon the order received by the customer (e.g., a customer can demand a 500 GB hard disk with an Intel processor; another customer can choose a 250 GB hard disk with an AMD processor).



Consequences

- Lets you vary a product's internal representation
- Isolates construction and representation
- Gives you finer control over the construction process



```
// Builders common interface
```

```
interface IBuilder
```

```
{
```

```
    void BuildBody();
```

```
    void InsertWheels();
```

```
    void AddHeadlights();
```

```
    Product GetVehicle();
```

```
}
```

```
// Car is ConcreteBuilder
```

```
class Car implements IBuilder
```

```
{
```

```
    private Product product = new Product();
```

```
    @Override
```

```
    public void BuildBody()
```

```
    {
```

```
        product.Add("This is a body of a Car");
```

```
    }
```

```
    @Override
```

```
    public void InsertWheels()
```

```
    {
```

```
        product.Add("4 wheels are added");
```

```
    }
```

```
    @Override
```

```
    public void AddHeadlights()
```

```
    {
```

```
        product.Add("2 Headlights are added");
```

```
    }
```

```
    public Product GetVehicle()
```

```
    {
```

```
        return product;
```

```
    }
```

```
}
```

```
// "Product"
```

```
class Product
```

```
{
```

```
    // We can use any data structure that you prefer. We have used
```

```
    private LinkedList<String> parts;
```

```
    public Product()
```

```
    {
```

```
        parts = new LinkedList<String>();
```

```
    }
```

```
    public void Add(String part)
```

```
    {
```

```
        //Adding parts
```

```
        parts.addLast(part);
```

```
    }
```

```
    public void Show()
```

```
    {
```

```
        System.out.println("\n Product completed as below :");
```

```
        for(int i=0;i<parts.size();i++)
```

```
        {
```

```
            System.out.println(parts.get(i));
```

```
        }
```

```
    }
```

```
}
```

```

// "Director"
class Director
{
    IBuilder myBuilder;

    // A series of steps—for the production
    public void Construct(IBuilder builder)
    {
        myBuilder=builder;
        myBuilder.BuildBody();
        myBuilder.InsertWheels();
        myBuilder.AddHeadlights();
    }
}

class BuilderPatternEx
{
    public static void main(String[] args)
    {
        System.out.println("***Builder Pattern Demo***\n");

        Director director = new Director();

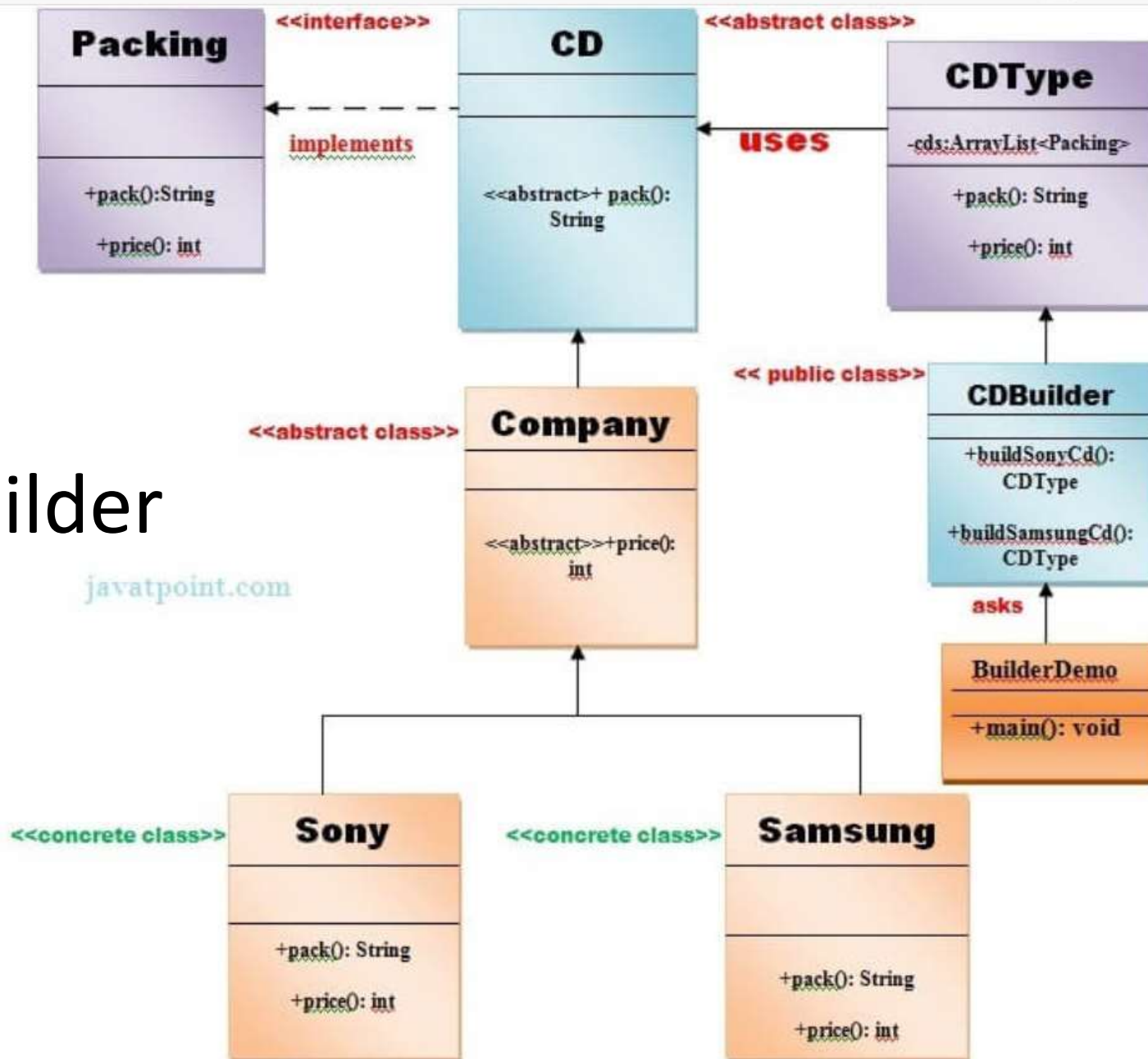
        IBuilder carBuilder = new Car();
        IBuilder motorBuilder = new MotorCycle();

        // Making Car
        director.Construct(carBuilder);
        Product p1 = carBuilder.GetVehicle();
        p1.Show();

        //Making MotorCycle
        director.Construct(motorBuilder);
        Product p2 = motorBuilder.GetVehicle();
        p2.Show();
    }
}

```

Builder



javatpoint.com

File: Packing.java

```
public interface Packing {  
    public String pack();  
    public int price();  
}
```

2) Create 2 abstract classes CD and Company

Create an abstract class CD which will implement Packing interface.

File: CD.java

```
public abstract class CD implements Packing{  
    public abstract String pack();  
}
```

File: Company.java

```
public abstract class Company extends CD{  
    public abstract int price();  
}
```

File: Sony.java

```
public class Sony extends Company{
    @Override
    public int price(){
        return 20;
    }
    @Override
    public String pack(){
        return "Sony CD";
    }
} //End of the Sony class.
```

File: Samsung.java

```
public class Samsung extends Company {
    @Override
    public int price(){
        return 15;
    }
    @Override
    public String pack(){
        return "Samsung CD";
    }
} //End of the Samsung class.
```

File: CDType.java

```
import java.util.ArrayList;
import java.util.List;
public class CDType {
    private List<Packing> items=new ArrayList<Packing>();
    public void addItem(Packing packs) {
        items.add(packs);
    }
    public void getCost(){
        for (Packing packs : items) {
            packs.price();
        }
    }
    public void showItems(){
        for (Packing packing : items){
            System.out.print("CD name : "+packing.pack());
            System.out.println(", Price : "+packing.price());
        }
    }
} //End of the CDType class.
```

File: CDBuilder.java

```
public class CDBuilder {  
    public CDType buildSonyCD(){  
        CDType cds=new CDType();  
        cds.addItem(new Sony());  
        return cds;  
    }  
    public CDType buildSamsungCD(){  
        CDType cds=new CDType();  
        cds.addItem(new Samsung());  
        return cds;  
    }  
} // End of the CDBuilder class.
```

6) Create the BuilderDemo class

File: BuilderDemo.java

```
public class BuilderDemo{  
    public static void main(String args[]){  
        CDBuilder cdBuilder=new CDBuilder();  
        CDType cdType1=cdBuilder.buildSonyCD();  
        cdType1.showItems();  
  
        CDType cdType2=cdBuilder.buildSamsungCD();  
        cdType2.showItems();  
    }  
}
```